

Synapse Hub — System Design Document

Status: Ready to Deploy **Owner:** Somasekhar Thupakula **Last updated:** 21-07-26

1. Executive Summary

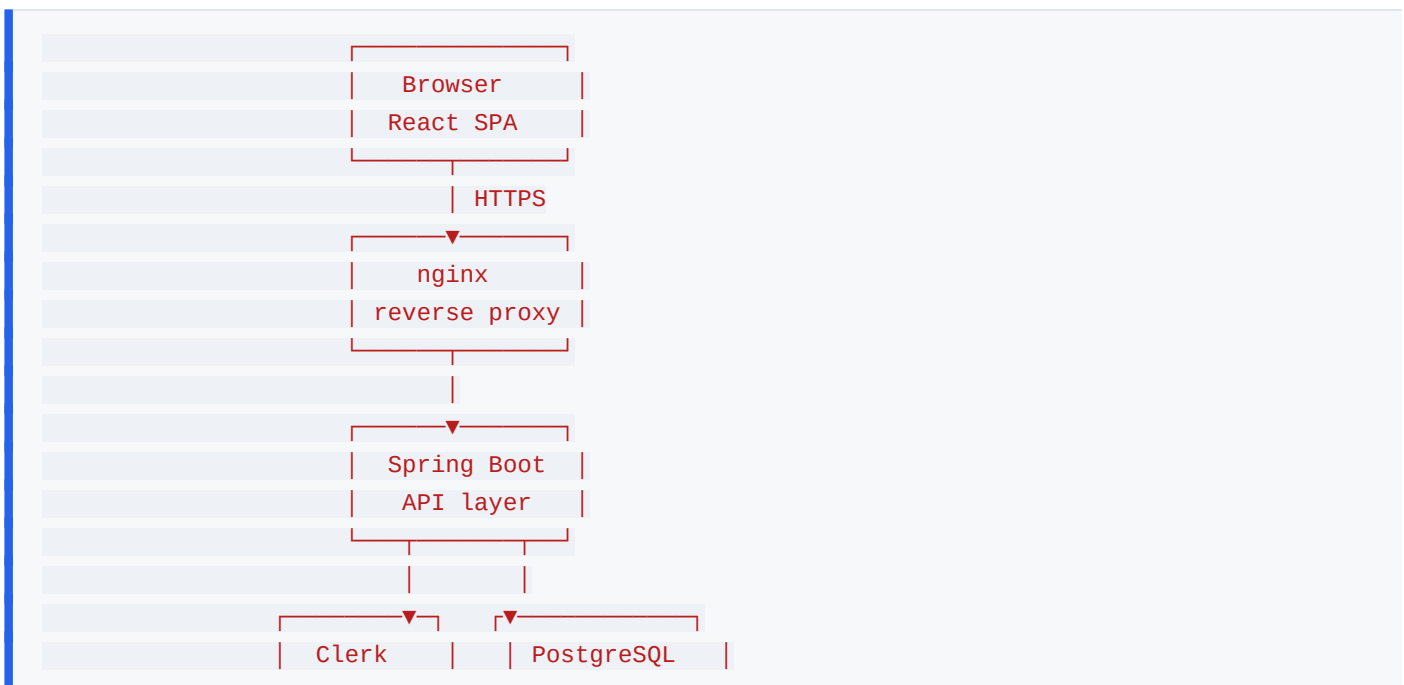
Synapse Hub is a multi-tenant internal idea management platform that gives organizations a structured, Reddit-style space for employees to submit ideas, discuss them through threaded comments, surface the best ones through a hot/new/top ranking system, and track them through a formal review lifecycle from submission to implementation.

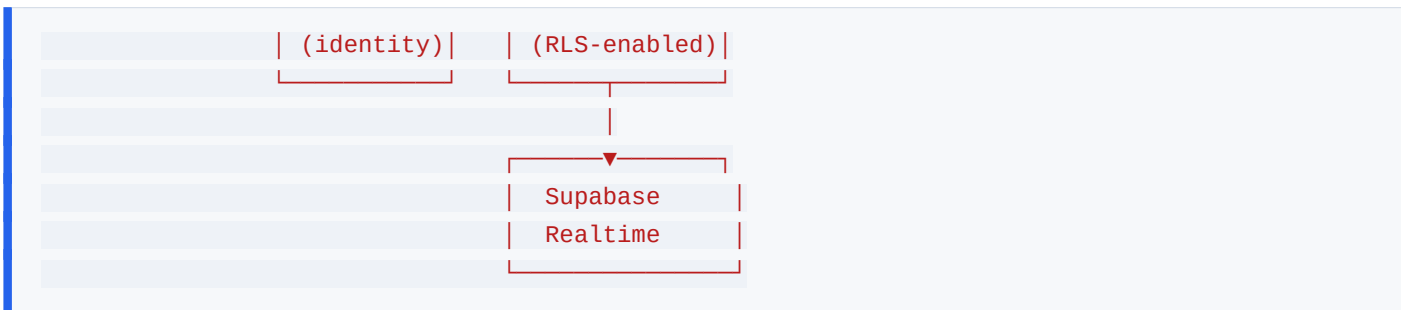
The system is designed around one central engineering constraint: every organization's data must be fully isolated from every other organization's, enforced redundantly at both the application and database layers, so that no single coding mistake can result in a cross-tenant data leak.

Key capabilities:

- Organization-scoped workspaces with role-based membership (admin/member)
- Idea submission, threaded commenting, and upvote/downvote with live updates
- Hot/New/Top ranking using a vote-and-recency decay algorithm
- Admin-controlled status lifecycle (Submitted → Under Review → Planned → Implemented/Rejected)
- Defense-in-depth tenant isolation via application-layer scoping and PostgreSQL Row-Level Security

2. System Architecture Overview





Requests enter through nginx, which terminates TLS and forwards to the Spring Boot API. The API verifies identity via Clerk-issued JWTs, applies application-level authorization scoped to the caller's organization, and reads/writes PostgreSQL — where Row-Level Security independently re-enforces that same organization boundary. Data changes propagate to connected clients via Supabase Realtime, itself governed by the same RLS policies.

3. Component Breakdown

3.1 Frontend — React / Vite SPA

A single-page application handling authentication state (via Clerk's React SDK), the idea feed, submission forms, threaded comments, and admin controls.

Key pages:

- Sign in / Organization selection
- Idea feed (Hot / New / Top tabs)
- Idea detail (comments, voting)
- Submit idea
- Admin dashboard (status management, member roles)

3.2 Backend — Spring Boot (Java)

Stateless REST API responsible for request validation, authorization enforcement, business logic (ranking calculation, vote integrity), and orchestration of real-time change propagation. Organized by feature ([org/](#), [idea/](#), [vote/](#), [comment/](#)), consistent with the package-by-feature structure used across the other two projects.

3.3 Database — PostgreSQL

Source of truth for organizations, users, ideas, votes, and comments. Row-Level Security policies are defined per-table and scoped to the authenticated caller's organization, acting as an independent enforcement layer beneath the application code. *(Confirm and update the version number once your deployment target is fixed — the header text assumed PostgreSQL 17 templating, verify this matches what you're actually running.)*

3.4 Reverse Proxy — nginx

Terminates TLS, forwards requests to the Spring Boot backend, and serves the built React SPA's static assets. Also the layer where core security headers (CSP, HSTS, X-Frame-Options) are applied.

4. Data Model

Core Entities

```
organizations
  id, name, created_at

memberships
  id, org_id (FK), user_id, role [ADMIN|MEMBER]

ideas
  id, org_id (FK), author_id, title, description, tag, status
  [SUBMITTED|UNDER_REVIEW|PLANNED|IMPLEMENTED|REJECTED], created_at

votes
  id, idea_id (FK), user_id, value [+1|-1]
  unique constraint: (idea_id, user_id)

comments
  id, idea_id (FK), author_id, parent_comment_id (nullable, for threading), body, created_at
```

Key Relationships

- An **organization** has many **memberships**, each tying one **user** to one **role**
- An **idea** belongs to exactly one **organization** and one author
- A **vote** belongs to exactly one **idea** and one user, with a unique constraint preventing duplicate votes
- A **comment** belongs to an **idea** and optionally to a parent **comment**, forming a thread

5. API Design

POST	/api/orgs	– create organization
POST	/api/orgs/join	– join via invite code
GET	/api/ideas?sort=hot new top	– list ideas for caller's org
POST	/api/ideas	– submit idea
GET	/api/ideas/{id}	– idea detail + comments
POST	/api/ideas/{id}/votes	– cast/change vote
DELETE	/api/ideas/{id}/votes	– remove vote
POST	/api/ideas/{id}/comments	– add comment
PATCH	/api/ideas/{id}/status	– admin-only status transition
GET	/api/orgs/members	– admin-only member/role list

Every endpoint derives **org_id** from the verified Clerk session — never from the request path, body, or query parameters — before touching the database.

6. Authentication & Authorization

6.1 Local Authentication (Bootstrap / Temporary)

For early local development before Clerk is fully wired in, a mock JWT sandbox mode issues locally-signed tokens with a fixed test org/user, letting the API and frontend be developed and tested without live Clerk credentials. This mode is explicitly disabled outside local/dev environments.

6.2 OAuth2 Authorization Code Flow (Production Auth)

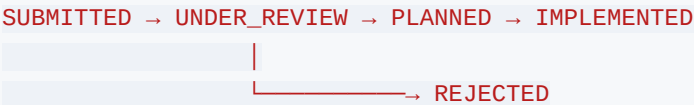
Clerk handles the full OAuth2/OIDC flow for real users, issuing signed JWTs containing user identity and organization membership claims. The backend verifies signature, issuer, audience, and expiry on every request.

6.3 (numbering follows the source outline as given — no 6.3 section was specified)

6.4 Role Permissions Summary

Action	Member	Admin
View ideas, vote, comment	✓	✓
Submit idea	✓	✓
Change idea status	✗	✓
Manage member roles	✗	✓
View org-wide member list	✗	✓

7. Idea Lifecycle



An idea enters **SUBMITTED** on creation. Only an org admin can move it forward; transitions are one-directional in the common path, with **REJECTED** reachable from **SUBMITTED** or **UNDER_REVIEW** directly. Every transition is recorded in an append-only audit log (who changed it, when, from what state to what state).

8. Infrastructure & Deployment

8.1 Environment

(Fill in once finalized — e.g. Railway/Render for the API, Vercel for the SPA, or self-hosted behind nginx on a single VM, consistent with the deployment reasoning used in your other two projects.)

8.2 Container Configuration

Backend and frontend each have a Dockerfile; local development and multi-container orchestration run via Docker Compose (API, Postgres, nginx).

8.3 Networking

nginx is the single public entry point; the Spring Boot API and PostgreSQL are not directly internet-exposed.

8.4 Secrets Management

Clerk keys, database credentials, and any third-party API keys are injected via environment variables at deploy time, never committed to source. *(Name your actual secrets manager/CI secret store once decided.)*

8.5 Database Backup

(Fill in the actual backup cadence and mechanism once configured — e.g. automated daily snapshots via your hosting provider — do not leave this section unaddressed at launch, see SECURITY-CHECKLIST.md.)

8.6 Deployment Process

GitHub Actions gates every merge to `main` on build, unit tests, and the security checks defined in the project's security checklist (dependency scan, SAST, secret scan) before triggering a deploy.

9. Security Design

Tenant isolation is enforced at two independent layers: application-level scoping (every query filtered by `org_id` derived from the verified session) and PostgreSQL Row-Level Security (an independent enforcement layer beneath the application, so a missed `WHERE` clause cannot alone cause a cross-tenant leak). Full threat model, defense-in-depth breakdown, and the pre-launch verification checklist are maintained separately — see `ARCHITECTURE.md` and `SECURITY-CHECKLIST.md`.

10. Non-Functional Requirements

- **Tenant isolation:** zero cross-org data exposure, verified via explicit IDOR and RLS tests, not assumed
- **Real-time latency:** vote and comment updates visible to other clients within a low, sub-second window under normal load
- **Availability:** a failure in the AI/ranking computation must not block core read/write availability of ideas and votes
- **Auditability:** every status change and role change is logged and attributable

11. Known Limitations & Future Considerations

- No duplicate-idea detection yet — near-identical ideas can be submitted independently
- No notification system — users must revisit the feed to see updates
- Ranking recalculation is not yet benchmarked at scale beyond the assumptions used in load testing

- Local/mock auth mode must be verified as fully unreachable in any deployed environment before go-live